

LAPORAN PRAKTIKUM
STRUKTUR DATA
BINARY TREE DAN GRAPH TRAVERSAL



Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang binary tree dan graph traversal menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa laporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep binary tree dan graph traversal.

Padang, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Dasar Teori	3
2.2 Kode Program	3
2.2.1 Class Node_2511533008	3
2.2.2 Class BTree_2511533008	6
2.2.3 Class BTreeDriver_2511533008.....	9
2.2.4 Class GraphTraversal_2511533008	11
2.3 Output Program.....	15
2.3.1 Output BTreeDriver_2511533008	15
2.3.2 Output GraphTraversal_2511533008.....	16
BAB III PENUTUP	17
3.1 Kesimpulan	17
DAFTAR ISI.....	18
TUGAS	19
1. Kode Program	19
2. Output Tugas.....	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan ilmu komputer, pemahaman terhadap struktur data non-linear menjadi hal yang sangat penting karena banyak digunakan dalam berbagai aplikasi kompleks. Salah satu bentuk struktur data tersebut adalah Binary Tree, yaitu pohon biner di mana setiap simpul memiliki maksimal dua anak. Binary Tree banyak dimanfaatkan dalam proses pencarian, pengurutan, serta representasi ekspresi matematika karena sifat hierarkisnya yang memudahkan pengelolaan data. Selain itu, konsep Graph Traversal juga menjadi fokus utama dalam praktikum ini. Graph Traversal merupakan teknik untuk menelusuri simpul-simpul dalam graf, dengan dua metode populer yaitu Breadth-First Search (BFS) yang menelusuri secara melebar dan Depth-First Search (DFS) yang menelusuri secara mendalam. Kedua algoritma ini memiliki peran penting dalam pencarian jalur, analisis jaringan, hingga pengembangan sistem kecerdasan buatan. Melalui praktikum ini, mahasiswa diharapkan mampu memahami konsep dasar Binary Tree dan Graph Traversal, serta mengimplementasikan algoritma BFS dan DFS untuk menyelesaikan permasalahan nyata secara efisien.

1.2 Tujuan

1. **Memahami konsep Binary Tree** → mengenali struktur pohon biner, sifat-sifatnya, serta penerapannya dalam pengolahan data.
2. **Mengimplementasikan Graph Traversal** → melatih penggunaan algoritma penelusuran simpul pada graf.
3. **Menerapkan algoritma BFS** → menelusuri graf secara melebar untuk memahami pencarian jalur dan keterhubungan simpul.
4. **Menerapkan algoritma DFS** → menelusuri graf secara mendalam untuk memahami jalur alternatif dan struktur graf.

5. **Menganalisis perbedaan BFS dan DFS** → membandingkan efisiensi, kelebihan, dan kekurangan masing-masing algoritma.
6. **Melatih keterampilan pemecahan masalah** → memilih algoritma yang tepat sesuai dengan kebutuhan kasus nyata.
7. **Menghubungkan teori dengan praktik** → membangun program sederhana yang merepresentasikan Binary Tree dan Graph Traversal.

1.3 Manfaat

1. **Pemahaman struktur data non-linear** → mahasiswa memperoleh wawasan tentang cara kerja pohon biner dan graf dalam mengorganisasi data.
2. **Keterampilan implementasi algoritma** → melatih kemampuan mengaplikasikan algoritma BFS dan DFS dalam program nyata.
3. **Analisis efisiensi algoritma** → memahami kelebihan dan kekurangan masing-masing metode traversal untuk kasus berbeda.
4. **Pengembangan kemampuan problem solving** → meningkatkan keterampilan memilih strategi yang tepat dalam menyelesaikan permasalahan komputasi.
5. **Penghubungan teori dengan praktik** → memberikan pengalaman langsung dalam mengimplementasikan konsep yang dipelajari di kelas.
6. **Bekal untuk aplikasi lanjutan** → mendukung pemahaman di bidang data mining, jaringan komputer, dan kecerdasan buatan.
7. **Peningkatan logika pemrograman** → memperkuat dasar berpikir algoritmik yang berguna untuk mata kuliah lanjutan maupun penelitian.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Dalam ilmu komputer, struktur data merupakan cara untuk menyimpan dan mengorganisasi data agar dapat diakses serta diproses secara efisien. Salah satu bentuk struktur data non-linear yang penting adalah Binary Tree, yaitu pohon biner di mana setiap simpul memiliki maksimal dua anak. Binary Tree digunakan dalam berbagai aplikasi, seperti algoritma pencarian, pengurutan, dan representasi ekspresi aritmatika. Dengan sifat hierarkisnya, Binary Tree memungkinkan proses penyimpanan dan pengambilan data dilakukan secara sistematis.

Selain itu, konsep Graph Traversal juga menjadi bagian penting dalam studi struktur data. Traversal pada graf adalah proses menelusuri simpul-simpul untuk menemukan informasi atau jalur tertentu. Dua algoritma utama yang digunakan adalah Breadth-First Search (BFS), yang menelusuri graf secara melebar dengan mengunjungi simpul-simpul tetangga terlebih dahulu, serta Depth-First Search (DFS), yang menelusuri graf secara mendalam dengan mengikuti satu jalur hingga akhir sebelum kembali ke simpul sebelumnya.

Kedua algoritma traversal ini memiliki peran penting dalam berbagai aplikasi nyata, seperti pencarian jalur terpendek, analisis jaringan, sistem rekomendasi, hingga pengembangan kecerdasan buatan. Dengan memahami dasar teori Binary Tree dan Graph Traversal, mahasiswa dapat menguasai konsep fundamental yang mendukung pemecahan masalah komputasi secara efisien dan terstruktur.

2.2 Kode Program

2.2.1 Class Node_2511533008

```
package pekan9_2511533008;  
  
public class Node_2511533008 {
```

```

int data_3008;
Node_2511533008 left_3008;
Node_2511533008 right_3008;
public Node_2511533008(int data_3008) {
    this.data_3008 = data_3008;
    left_3008 = null;
    right_3008 = null;
}
public void setLeft_3008(Node_2511533008 node_3008) {
    if (left_3008 == null)
        left_3008 = node_3008;
}
public void setRight_3008(Node_2511533008 node_3008) {
    if (right_3008 == null)
        right_3008 = node_3008;
}
public Node_2511533008 getLeft_3008() {
    return left_3008;
}
public Node_2511533008 getRight_3008() {
    return right_3008;
}
public int getData_3008() {
    return data_3008;
}
public void setData_3008(int data_3008) {
    this.data_3008 = data_3008;
}
void printPreorder_3008(Node_2511533008 node_3008) {
    if (node_3008 == null)
        return;
    System.out.print(node_3008.data_3008 + " ");
    printPreorder_3008(node_3008.left_3008);
    printPreorder_3008(node_3008.right_3008);
}
void printPostorder_3008(Node_2511533008 node_3008) {
    if (node_3008 == null)
        return;
    printPostorder_3008(node_3008.left_3008);
    printPostorder_3008(node_3008.right_3008);
    System.out.print(node_3008.data_3008 + " ");
}
void printInorder_3008(Node_2511533008 node_3008) {
    if (node_3008 == null)
        return;
    printInorder_3008(node_3008.left_3008);
    System.out.print(node_3008.data_3008 + " ");
    printInorder_3008(node_3008.right_3008);
}
public String print_3008() {
    return this.print_3008("", true, "");
}

```

```

    public String print_3008(String prefix_3008, boolean
isTail_3008, String sb_3008) {
        if (right_3008 != null) {
            right_3008.print_3008(prefix_3008 + (isTail_3008
? "|" : " "), false, sb_3008);
        }
        System.out.println(prefix_3008 + (isTail_3008 ? "\\--"
: "/--") + data_3008);
        if (left_3008 != null) {
            left_3008.print_3008(prefix_3008 + (isTail_3008
? " " : "| "), true, sb_3008);
        }
        return sb_3008;
    }
}

```

Penjelasan :

1. Konstruktor

```

public Node_2511533008(int data_3008) {
    this.data_3008 = data_3008;
    left_3008 = null;
    right_3008 = null;
}

```

Gambar 2.2.1.1 konstruktor

Dimana terdapat node baru dengan nilai data_3008 yang bagian kiri dan kanan bernilai null.

2. Setter dan getter

```

public void setLeft_3008(Node_2511533008 node_3008) {
    if (left_3008 == null)
        left_3008 = node_3008;
}
public void setRight_3008(Node_2511533008 node_3008) {
    if (right_3008 == null)
        right_3008 = node_3008;
}
public Node_2511533008 getLeft_3008() {
    return left_3008;
}
public Node_2511533008 getRight_3008() {
    return right_3008;
}
public int getData_3008() {
    return data_3008;
}
public void setData_3008(int data_3008) {
    this.data_3008 = data_3008;
}
}

```

Gambar 2.2.1.2 setter dan getter

3. Traversal Binary Tree

a. Preorder (root → left → right)

```
void printPreorder_3008(Node_2511533008 node_3008) {  
    if (node_3008 == null)  
        return;  
    System.out.print(node_3008.data_3008 + " ");  
    printPreorder_3008(node_3008.left_3008);  
    printPreorder_3008(node_3008.right_3008);  
}
```

Gambar 2.2.1.3 print preorder

b. Postorder (left → right → root)

```
void printPostorder_3008(Node_2511533008 node_3008) {  
    if (node_3008 == null)  
        return;  
    printPostorder_3008(node_3008.left_3008);  
    printPostorder_3008(node_3008.right_3008);  
    System.out.print(node_3008.data_3008 + " ");  
}
```

Gambar 2.2.1.4 print postorder

c. Inorder (left → root → right)

```
void printInorder_3008(Node_2511533008 node_3008) {  
    if (node_3008 == null)  
        return;  
    printInorder_3008(node_3008.left_3008);  
    System.out.print(node_3008.data_3008 + " ");  
    printInorder_3008(node_3008.right_3008);  
}
```

Gambar 2.2.1.5 print inorder

4. Membuat visualisasi pohon

```
public String print_3008() {  
    return this.print_3008("", true, "");  
}  
public String print_3008(String prefix_3008, boolean isTail_3008, String sb_3008) {  
    if (right_3008 != null) {  
        right_3008.print_3008(prefix_3008 + (isTail_3008 ? "| " : " "), false, sb_3008);  
    }  
    System.out.println(prefix_3008 + (isTail_3008 ? "\\--" : "/--") + data_3008);  
    if (left_3008 != null) {  
        left_3008.print_3008(prefix_3008 + (isTail_3008 ? " " : "| "), true, sb_3008);  
    }  
    return sb_3008;  
}
```

Gamabar 2.2.1.6 membuat visualisasi pohon

2.2.2 Class BTree_2511533008

```
package pekan9_2511533008;  
  
public class BTree_2511533008 {
```

```

private Node_2511533008 root_3008;
private Node_2511533008 currentNode_3008;
public BTree_2511533008() {
    root_3008 = null;
}
public boolean search_3008(int data_3008) {
    return search_3008(root_3008, data_3008);
}
private boolean search_3008(Node_2511533008 node_3008, int
data_3008) {
    if (node_3008.getData_3008() == data_3008)
        return true;
    if (node_3008.getLeft_3008() != null)
        if (search_3008(node_3008.getLeft_3008(),
data_3008))
            return true;
    if (node_3008.getRight_3008() != null)
        if (search_3008(node_3008.getRight_3008(),
data_3008))
            return true;
    return false;
}
public void printInorder_3008() {
    root_3008.printInorder_3008(root_3008);
}
public void printPreorder_3008() {
    root_3008.printPreorder_3008(root_3008);
}
public void printPostorder_3008() {
    root_3008.printPostorder_3008(root_3008);
}
public Node_2511533008 getRoot_3008() {
    return root_3008;
}
public boolean isEmpty_3008() {
    return root_3008 == null;
}
public int countNodes_3008() {
    return countNodes_3008(root_3008);
}
public int countNodes_3008(Node_2511533008 node_3008) {
    int count_3008 = 1;
    if (node_3008 == null) {
        return 0;
    } else {
        count_3008 +=
countNodes_3008(node_3008.getLeft_3008());
        count_3008 +=
countNodes_3008(node_3008.getRight_3008());
        return count_3008;
    }
}
public void print_3008() {

```

```

        root_3008.print_3008();
    }
    public Node_2511533008 getCurrent_3008() {
        return currentNode_3008;
    }
    public void setCurrent_3008(Node_2511533008 node_3008) {
        this.currentNode_3008 = node_3008;
    }
    public void setRoot_3008(Node_2511533008 root_3008) {
        this.root_3008 = root_3008;
    }
}

```

Penjelasan:

1. Pencarian data melalui search_3008

```

public boolean search_3008(int data_3008) {
    return search_3008(root_3008, data_3008);
}
private boolean search_3008(Node_2511533008 node_3008, int data_3008) {
    if (node_3008.getData_3008() == data_3008)
        return true;
    if (node_3008.getLeft_3008() != null)
        if (search_3008(node_3008.getLeft_3008(), data_3008))
            return true;
    if (node_3008.getRight_3008() != null)
        if (search_3008(node_3008.getRight_3008(), data_3008))
            return true;
    return false;
}

```

Gamabar 2.2.2.1 penjelasan 1-4

2. Mengecek apakah data_3008 ada di node saat ini.
3. Jika tidak, lanjut ke anak kiri dan kanan secara rekursif.
4. Mengembalikan true jika ditemukan, false jika tidak ada.
5. Traversal

```

public void printInorder_3008() {
    root_3008.printInorder_3008(root_3008);
}
public void printPreorder_3008() {
    root_3008.printPreorder_3008(root_3008);
}
public void printPostorder_3008() {
    root_3008.printPostorder_3008(root_3008);
}

```

Gambar 2.2.2.2 traversal

Memanggil fungsi traversal dari kelas Node_2511533008.

6. Mencek tree kosong dan Menghitung jumlah node

```
public boolean isEmpty_3008() {
    return root_3008 == null;
}
public int countNodes_3008() {
    return countNodes_3008(root_3008);
}
public int countNodes_3008(Node_2511533008 node_3008) {
    int count_3008 = 1;
    if (node_3008 == null) {
        return 0;
    } else {
        count_3008 += countNodes_3008(node_3008.getLeft_3008());
        count_3008 += countNodes_3008(node_3008.getRight_3008());
        return count_3008;
    }
}
```

Gambar 2.2.2.3

7. Visualisasi pohon

```
public void print_3008() {
    root_3008.print_3008();
}
```

Gambar 2.2.2.4

2.2.3 Class BTreeDriver_2511533008

```
package pekan9_2511533008;

public class BTreeDriver_2511533008 {
    public static void main(String[] args) {
        //membuat pohon
        BTree_2511533008 tree_3008 = new BTree_2511533008();
        System.out.print("Jumlah Simpul Awal Pohon: ");
        System.out.println(tree_3008.countNodes_3008());
        //menambahkan simpul data 1
        Node_2511533008 root_3008 = new Node_2511533008(1);
        //menjadikan simpul 1 sebagai root
        tree_3008.setRoot_3008(root_3008);
        System.out.println("Jumlah simpul jika hanya ada
root");
        System.out.println(tree_3008.countNodes_3008());
        Node_2511533008 node2_3008 = new Node_2511533008(2);
        Node_2511533008 node3_3008 = new Node_2511533008(3);
        Node_2511533008 node4_3008 = new Node_2511533008(4);
        Node_2511533008 node5_3008 = new Node_2511533008(5);
        Node_2511533008 node6_3008 = new Node_2511533008(6);
        Node_2511533008 node7_3008 = new Node_2511533008(7);
        Node_2511533008 node8_3008 = new Node_2511533008(8);
        Node_2511533008 node9_3008 = new Node_2511533008(9);
        root_3008.setLeft_3008(node2_3008);
```

```

        node2_3008.setLeft_3008(node4_3008);
        node2_3008.setRight_3008(node5_3008);
        node4_3008.setRight_3008(node8_3008);
        root_3008.setRight_3008(node3_3008);
        node3_3008.setLeft_3008(node6_3008);
        node3_3008.setRight_3008(node7_3008);
        node6_3008.setLeft_3008(node9_3008);

        //set root
        tree_3008.setCurrent_3008(tree_3008.getRoot_3008());
        System.out.println("Menampilkan simpul terakhir: ");

        System.out.println(tree_3008.getCurrent_3008().getData_3008())
;
        System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
        System.out.println(tree_3008.countNodes_3008());
        System.out.println("InOrder: ");
        tree_3008.printInorder_3008();
        System.out.println("\nPreOrder: ");
        tree_3008.printPreorder_3008();
        System.out.println("\nPostOrder: ");
        tree_3008.printPostorder_3008();
        System.out.println("\nDmenampilkan simpul dalam bentuk
pohon");
        tree_3008.print_3008();
    }
}

```

Penjelasan:

1. Membuat pohon

```

//membuat pohon
BTree_2511533008 tree_3008 = new BTree_2511533008();
System.out.print("Jumlah Simpul Awal Pohon: ");
System.out.println(tree_3008.countNodes_3008());

```

Gambar 2.2.3.1 membuat pohon

2. Menambahkan simpul data 1 dan menjadikannya sebagai root

```

//menambahkan simpul data 1
Node_2511533008 root_3008 = new Node_2511533008(1);
//menjadikan simpul 1 sebagai root
tree_3008.setRoot_3008(root_3008);
System.out.println("Jumlah simpul jika hanya ada root");
System.out.println(tree_3008.countNodes_3008());

```

Gambar 2.2.3.2 simpul data 1 root

3. Menambahkan node lainnya

```

Node_2511533008 node2_3008 = new Node_2511533008(2);
Node_2511533008 node3_3008 = new Node_2511533008(3);
Node_2511533008 node4_3008 = new Node_2511533008(4);
Node_2511533008 node5_3008 = new Node_2511533008(5);
Node_2511533008 node6_3008 = new Node_2511533008(6);
Node_2511533008 node7_3008 = new Node_2511533008(7);
Node_2511533008 node8_3008 = new Node_2511533008(8);
Node_2511533008 node9_3008 = new Node_2511533008(9);

```

Gambar 2.2.3.3 menambahkan node

4. Menyusun struktur pohon

```

root_3008.setLeft_3008(node2_3008);
node2_3008.setLeft_3008(node4_3008);
node2_3008.setRight_3008(node5_3008);
node4_3008.setRight_3008(node8_3008);
root_3008.setRight_3008(node3_3008);
node3_3008.setLeft_3008(node6_3008);
node3_3008.setRight_3008(node7_3008);
node6_3008.setLeft_3008(node9_3008);

```

Gambar 2.2.3.4 menyusun struktur pohon

5. Mengatur current node dan menghitung jumlah node

```

tree_3008.setCurrent_3008(tree_3008.getRoot_3008());
System.out.println("Menampilkan simpul terakhir: ");
System.out.println(tree_3008.getCurrent_3008().getData_3008());
System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
System.out.println(tree_3008.countNodes_3008());

```

Gambar 2.2.3.5

6. Menampilkan traversal dan visualisasi pohon

```

System.out.println("InOrder: ");
tree_3008.printInorder_3008();
System.out.println("\nPreOrder: ");
tree_3008.printPreorder_3008();
System.out.println("\nPostOrder: ");
tree_3008.printPostorder_3008();
System.out.println("\nMenampilkan simpul dalam bentuk pohon");
tree_3008.print_3008();

```

Gambar 2.2.3.6

2.2.4 Class GraphTraversal_2511533008

```

package pekan9_2511533008;

import java.util.*;

public class GraphTraversal_2511533008 {
    private Map<String, List<String>> graph_3008 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_3008(String node1_3008, String node2_3008) {

```

```

graph_3008.putIfAbsent(node1_3008, new ArrayList<>());
graph_3008.putIfAbsent(node2_3008, new ArrayList<>());
graph_3008.get(node1_3008).add(node2_3008);
graph_3008.get(node2_3008).add(node1_3008);
}

// Menampilkan graf awal
public void printGraph_3008() {
    System.out.println("Graf Awal (Adjacency List):");
    for (String node_3008 : graph_3008.keySet()) {
        System.out.print(node_3008 + " -> ");
        List<String> neighbors_3008 = graph_3008.get(node_3008);
        System.out.println(String.join(", ", neighbors_3008));
    }
    System.out.println();
}

// DFS rekursif
public void dfs_3008(String start_3008) {
    Set<String> visited_3008 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_3008(start_3008, visited_3008);
    System.out.println();
}

private void dfsHelper_3008(String current_3008, Set<String>
visited_3008) {
    if (visited_3008.contains(current_3008)) return;
    visited_3008.add(current_3008);
    System.out.print(current_3008 + " ");
    for (String neighbor_3008 :
graph_3008.getDefault(current_3008, new ArrayList<>())) {
        dfsHelper_3008(neighbor_3008, visited_3008);
    }
}

// BFS iteratif
public void bfs_3008(String start_3008) {
    Set<String> visited_3008 = new HashSet<>();
    Queue<String> queue_3008 = new LinkedList<>();
    queue_3008.add(start_3008);
    visited_3008.add(start_3008);
    System.out.println("Penelusuran BFS:");
    while (!queue_3008.isEmpty()) {
        String current_3008 = queue_3008.poll();
        System.out.print(current_3008 + " ");
        for (String neighbor_3008 :
graph_3008.getDefault(current_3008, new ArrayList<>())) {
            if (!visited_3008.contains(neighbor_3008)) {
                queue_3008.add(neighbor_3008);
                visited_3008.add(neighbor_3008);
            }
        }
    }
}

```

```

        System.out.println();
    }

    // Main
    public static void main(String[] args) {
        GraphTraversal_2511533008 graph_3008 = new
GraphTraversal_2511533008();

        // Contoh graf: A-B, A-C, B-D, B-E
        graph_3008.addEdge_3008("A", "B");
        graph_3008.addEdge_3008("A", "C");
        graph_3008.addEdge_3008("B", "D");
        graph_3008.addEdge_3008("B", "E");

        // Cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_3008.printGraph_3008();

        // Lakukan penelusuran
        graph_3008.dfs_3008("A");
        graph_3008.bfs_3008("A");
    }
}

```

Penjelasan:

1. membuat struktur data graph
2. menambahkan edge

```

// Menambahkan edge (graf tak berarah)
public void addEdge_3008(String node1_3008, String node2_3008) {
    graph_3008.putIfAbsent(node1_3008, new ArrayList<>());
    graph_3008.putIfAbsent(node2_3008, new ArrayList<>());
    graph_3008.get(node1_3008).add(node2_3008);
    graph_3008.get(node2_3008).add(node1_3008);
}

```

Gambar 2.2.4.1

3. menampilkan graph awal

```
// Menampilkan graf awal
public void printGraph_3008() {
    System.out.println("Graf Awal (Adjacency List):");
    for (String node_3008 : graph_3008.keySet()) {
        System.out.print(node_3008 + " -> ");
        List<String> neighbors_3008 = graph_3008.get(node_3008);
        System.out.println(String.join(", ", neighbors_3008));
    }
    System.out.println();
}
```

Gambar 2.2.4.2

4. DFS

```
// DFS rekursif
public void dfs_3008(String start_3008) {
    Set<String> visited_3008 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_3008(start_3008, visited_3008);
    System.out.println();
}

private void dfsHelper_3008(String current_3008, Set<String> visited_3008) {
    if (visited_3008.contains(current_3008)) return;
    visited_3008.add(current_3008);
    System.out.print(current_3008 + " ");
    for (String neighbor_3008 : graph_3008.getDefault(current_3008, new ArrayList<>())) {
        dfsHelper_3008(neighbor_3008, visited_3008);
    }
}
```

Gambar 2.2.4.3

5. BFS

```
// BFS iteratif
public void bfs_3008(String start_3008) {
    Set<String> visited_3008 = new HashSet<>();
    Queue<String> queue_3008 = new LinkedList<>();
    queue_3008.add(start_3008);
    visited_3008.add(start_3008);
    System.out.println("Penelusuran BFS:");
    while (!queue_3008.isEmpty()) {
        String current_3008 = queue_3008.poll();
        System.out.print(current_3008 + " ");
        for (String neighbor_3008 : graph_3008.getDefault(current_3008, new ArrayList<>())) {
            if (!visited_3008.contains(neighbor_3008)) {
                queue_3008.add(neighbor_3008);
                visited_3008.add(neighbor_3008);
            }
        }
    }
    System.out.println();
}
```

Gambar 2.2.4.4

6. Fungsi main (algoritma utama) → membuat graph dan melakukan penelusuran DFS dan BFS.

```
// Main
public static void main(String[] args) {
    GraphTraversal_2511533008 graph_3008 = new GraphTraversal_2511533008();

    // Contoh graf: A-B, A-C, B-D, B-E
    graph_3008.addEdge_3008("A", "B");
    graph_3008.addEdge_3008("A", "C");
    graph_3008.addEdge_3008("B", "D");
    graph_3008.addEdge_3008("B", "E");

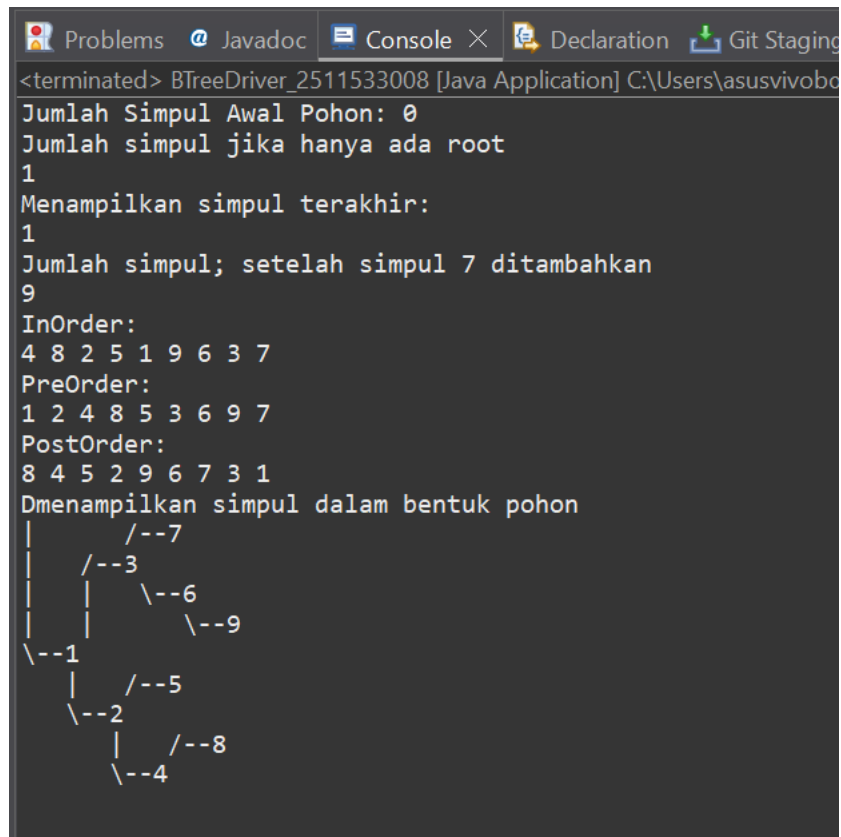
    // Cetak graf awal
    System.out.println("Graf Awal adalah: ");
    graph_3008.printGraph_3008();

    // Lakukan penelusuran
    graph_3008.dfs_3008("A");
    graph_3008.bfs_3008("A");
}
```

Gambar 2.2.4.5 main program

2.3 Output Program

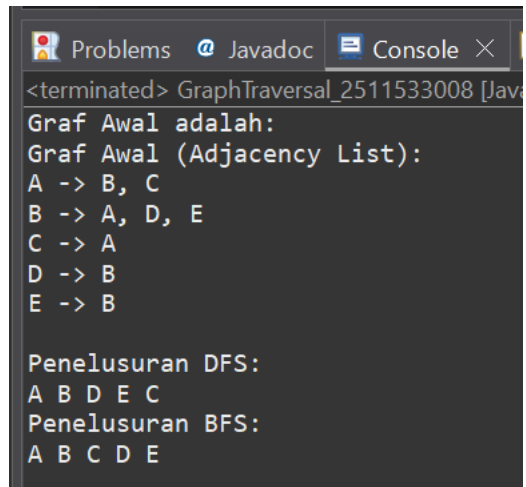
2.3.1 Output BTreeDriver_2511533008



```
<terminated> BTreeDriver_2511533008 [Java Application] C:\Users\asusvivobc
Jumlah Simpul Awal Pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|      /--7
|      /--3
|      |      \--6
|      |      \--9
|--1
|      /--5
|--2
|      /--8
|--4
```

Gambar 2.3.1 output BTreeDriver

2.3.2 Output GraphTraversal_2511533008



```
<terminated> GraphTraversal_2511533008 [Java]
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 2.3.2 output graph traversal

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa Binary Tree merupakan salah satu struktur data non-linear yang efektif dalam menyimpan dan mengorganisasi data secara hierarkis. Melalui implementasi pohon biner, mahasiswa dapat memahami bagaimana proses pencarian, pengurutan, dan representasi data dilakukan dengan lebih terstruktur.

Selain itu, praktikum juga menunjukkan bahwa Graph Traversal memiliki peran penting dalam menelusuri simpul-simpul graf untuk menemukan jalur atau informasi tertentu. Algoritma Breadth-First Search (BFS) menelusuri graf secara melebar, sedangkan Depth-First Search (DFS) menelusuri graf secara mendalam. Keduanya memiliki kelebihan dan kekurangan masing-masing, sehingga pemilihan algoritma harus disesuaikan dengan kebutuhan permasalahan.

Dengan demikian, praktikum ini memberikan pemahaman mendalam mengenai penerapan struktur data non-linear, melatih keterampilan analisis algoritma, serta memperkuat dasar logika pemrograman yang akan sangat berguna dalam pengembangan aplikasi maupun penelitian di bidang komputer.

DAFTAR ISI

- [1] Universitas Andalas, "Binary Tree," Modul Struktur Data dan Algoritma, Pekan 13, 2026.
- [2] Universitas Andalas, "Struktur Data Graf," Modul Struktur Data dan Algoritma, Pekan 14, 2026.
- [3] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.
- [4] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.

TUGAS

1. Kode Program

```
package pekan9_2511533008;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;

public class petaKampus_2511533008 extends JFrame {
    private Map<String, List<String>> graph_3008 = new HashMap<>();
    private JComboBox<String> startBox_3008, goalBox_3008;
    private JTextArea graphArea_3008, resultArea_3008;
    private JLabel pathLabel_3008, visitedLabel_3008, countLabel_3008;

    public petaKampus_2511533008() {
        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
        setSize(700, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        graph_3008.put("Gedung A", Arrays.asList("Gedung B", "Gedung C"));
        graph_3008.put("Gedung B", Arrays.asList("Gedung A", "Gedung D",
"Gedung E"));
        graph_3008.put("Gedung C", Arrays.asList("Gedung A", "Gedung F"));
        graph_3008.put("Gedung D", Arrays.asList("Gedung B", "Gedung G"));
        graph_3008.put("Gedung E", Arrays.asList("Gedung B", "Gedung H"));
        graph_3008.put("Gedung F", Arrays.asList("Gedung C", "Gedung I"));
        graph_3008.put("Gedung G", Arrays.asList("Gedung D", "Gedung J"));
        graph_3008.put("Gedung H", Arrays.asList("Gedung E"));
        graph_3008.put("Gedung I", Arrays.asList("Gedung F"));
        graph_3008.put("Gedung J", Arrays.asList("Gedung G"));

        // --- Panel kontrol ---
        JPanel controlPanel_3008 = new JPanel();
        startBox_3008 = new JComboBox<>(graph_3008.keySet().toArray(new
String[0]));
        goalBox_3008 = new JComboBox<>(graph_3008.keySet().toArray(new
String[0]));

        JButton bfsBtn_3008 = new JButton("BFS");
        bfsBtn_3008.setBackground(Color.GREEN);
        JButton dfsBtn_3008 = new JButton("DFS");
        dfsBtn_3008.setBackground(Color.ORANGE);
        JButton resetBtn_3008 = new JButton("RESET");
        resetBtn_3008.setBackground(Color.RED);

        controlPanel_3008.add(new JLabel("Lokasi Awal:"));
        controlPanel_3008.add(startBox_3008);
    }
}
```

```

        controlPanel_3008.add(new JLabel("Lokasi Tujuan:"));
        controlPanel_3008.add(goalBox_3008);
        controlPanel_3008.add(bfsBtn_3008);
        controlPanel_3008.add(dfsBtn_3008);
        controlPanel_3008.add(resetBtn_3008);

        add(controlPanel_3008, BorderLayout.NORTH);

        // --- Area visualisasi graph ---
        graphArea_3008 = new JTextArea();
        graphArea_3008.setEditable(false);

graphArea_3008.setBorder(BorderFactory.createTitledBorder("VISUALISASI
GRAPH"));
        tampilkanGraph_3008();

        // --- Area hasil pencarian ---
        JPanel resultPanel_3008 = new JPanel(new GridLayout(3,1));
        pathLabel_3008 = new JLabel("Jalur: -");
        visitedLabel_3008 = new JLabel("Node Dikunjungi: -");
        countLabel_3008 = new JLabel("Jumlah Node Dikunjungi: 0");
        resultPanel_3008.setBorder(BorderFactory.createTitledBorder("Hasil
Pencarian"));
        resultPanel_3008.add(pathLabel_3008);
        resultPanel_3008.add(visitedLabel_3008);
        resultPanel_3008.add(countLabel_3008);

        JSplitPane splitPane_3008 = new
JSplitPane(JSplitPane.VERTICAL_SPLIT,
            new JScrollPane(graphArea_3008), resultPanel_3008);
        splitPane_3008.setDividerLocation(250);
        add(splitPane_3008, BorderLayout.CENTER);

        // --- Action ---
        bfsBtn_3008.addActionListener(e -> jalankanBFS_3008());
        dfsBtn_3008.addActionListener(e -> jalankanDFS_3008());
        resetBtn_3008.addActionListener(e -> resetOutput_3008());
    }

    private void tampilkanGraph_3008() {
        StringBuilder sb_3008 = new StringBuilder();
        for (String node_3008 : graph_3008.keySet()) {
            sb_3008.append(node_3008).append(" ->
").append(graph_3008.get(node_3008)).append("\n");
        }
        graphArea_3008.setText(sb_3008.toString());
    }

    private void jalankanBFS_3008() {
        String start_3008 = (String) startBox_3008.getSelectedItem();
        String goal_3008 = (String) goalBox_3008.getSelectedItem();

        Queue<String> queue_3008 = new LinkedList<>();

```

```

        Set<String> visited_3008 = new LinkedHashSet<>();
        Map<String, String> parent_3008 = new HashMap<>();

        queue_3008.add(start_3008);
        visited_3008.add(start_3008);

        while (!queue_3008.isEmpty()) {
            String current_3008 = queue_3008.poll();
            if (current_3008.equals(goal_3008)) break;

            for (String neighbor_3008 :
graph_3008.getDefault(current_3008, new ArrayList<>())) {
                if (!visited_3008.contains(neighbor_3008)) {
                    visited_3008.add(neighbor_3008);
                    parent_3008.put(neighbor_3008, current_3008);
                    queue_3008.add(neighbor_3008);
                }
            }
        }
        tampilkanHasil_3008(parent_3008, visited_3008, start_3008,
goal_3008);
    }

    private void jalankanDFS_3008() {
        String start_3008 = (String) startBox_3008.getSelectedItem();
        String goal_3008 = (String) goalBox_3008.getSelectedItem();

        Stack<String> stack_3008 = new Stack<>();
        Set<String> visited_3008 = new LinkedHashSet<>();
        Map<String, String> parent_3008 = new HashMap<>();

        stack_3008.push(start_3008);

        while (!stack_3008.isEmpty()) {
            String current_3008 = stack_3008.pop();
            if (!visited_3008.contains(current_3008)) {
                visited_3008.add(current_3008);
                if (current_3008.equals(goal_3008)) break;

                for (String neighbor_3008 :
graph_3008.getDefault(current_3008, new ArrayList<>())) {
                    if (!visited_3008.contains(neighbor_3008)) {
                        parent_3008.put(neighbor_3008, current_3008);
                        stack_3008.push(neighbor_3008);
                    }
                }
            }
        }
        tampilkanHasil_3008(parent_3008, visited_3008, start_3008,
goal_3008);
    }

```

```

        private void tampilkanHasil_3008(Map<String,String> parent_3008,
        Set<String> visited_3008,
                                String start_3008, String goal_3008) {
            List<String> path_3008 = new ArrayList<>();
            String current_3008 = goal_3008;
            while (current_3008 != null) {
                path_3008.add(current_3008);
                current_3008 = parent_3008.get(current_3008);
            }
            Collections.reverse(path_3008);

            pathLabel_3008.setText("Jalur: " + path_3008);
            visitedLabel_3008.setText("Node Dikunjungi: " + visited_3008);
            countLabel_3008.setText("Jumlah Node Dikunjungi: " +
visited_3008.size());
        }

        private void resetOutput_3008() {
            pathLabel_3008.setText("Jalur: -");
            visitedLabel_3008.setText("Node Dikunjungi: -");
            countLabel_3008.setText("Jumlah Node Dikunjungi: 0");
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> new
petaKampus_2511533008().setVisible(true));
        }
    }
}

```

2. Output Tugas

PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS

Lokasi Awal: Gedung I Lokasi Tujuan: Gedung I **BFS** **DFS** **RESET**

VISUALISASI GRAPH

Gedung I -> [Gedung F]
Gedung H -> [Gedung E]
Gedung J -> [Gedung G]
Gedung A -> [Gedung B, Gedung C]
Gedung C -> [Gedung A, Gedung F]
Gedung B -> [Gedung A, Gedung D, Gedung E]
Gedung E -> [Gedung B, Gedung H]
Gedung D -> [Gedung B, Gedung G]
Gedung G -> [Gedung D, Gedung J]
Gedung F -> [Gedung C, Gedung I]

Hasil Pencarian

Jalur: -

Node Dikunjungi: -

Jumlah Node Dikunjungi: 0

Tampilan awal

PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS

Lokasi Awal: Gedung I Lokasi Tujuan: Gedung I **BFS** **DFS** **RESET**

VISUALISASI GRAPH

Gedung I -> [Gedung F]
Gedung H -> [Gedung E]
Gedung J -> [Gedung G]
Gedung A -> [Gedung B, Gedung C]
Gedung C -> [Gedung A, Gedung F]
Gedung B -> [Gedung A, Gedung D, Gedung E]
Gedung E -> [Gedung B, Gedung H]
Gedung D -> [Gedung B, Gedung G]
Gedung G -> [Gedung D, Gedung J]
Gedung F -> [Gedung C, Gedung I]

Hasil Pencarian

Jalur: -

Node Dikunjungi: -

Jumlah Node Dikunjungi: 0

Pilih lokasi awal

PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS

Lokasi Awal: Lokasi Tujuan:

VISUALISASI GRAPH

Gedung I -> [Gedung F]
 Gedung H -> [Gedung E]
 Gedung J -> [Gedung G]
 Gedung A -> [Gedung B, Gedung C]
 Gedung C -> [Gedung A, Gedung F]
 Gedung B -> [Gedung A, Gedung D, Gedung E]
 Gedung E -> [Gedung B, Gedung H]
 Gedung D -> [Gedung B, Gedung G]
 Gedung G -> [Gedung D, Gedung J]
 Gedung F -> [Gedung C, Gedung I]

Hasil Pencarian

Jalur: -

Node Dikunjungi: -

Jumlah Node Dikunjungi: 0

Pilih lokasi tujuan

PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS

Lokasi Awal: Lokasi Tujuan:

VISUALISASI GRAPH

Gedung I -> [Gedung F]
 Gedung H -> [Gedung E]
 Gedung J -> [Gedung G]
 Gedung A -> [Gedung B, Gedung C]
 Gedung C -> [Gedung A, Gedung F]
 Gedung B -> [Gedung A, Gedung D, Gedung E]
 Gedung E -> [Gedung B, Gedung H]
 Gedung D -> [Gedung B, Gedung G]
 Gedung G -> [Gedung D, Gedung J]
 Gedung F -> [Gedung C, Gedung I]

Hasil Pencarian

Jalur: [Gedung A, Gedung C]

Node Dikunjungi: [Gedung A, Gedung B, Gedung C, Gedung D, Gedung E]

Jumlah Node Dikunjungi: 5

Pencarian melalui BFS

PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS

Lokasi Awal: Gedung A Lokasi Tujuan: Gedung C **BFS** **DFS** **RESET**

VISUALISASI GRAPH

Gedung I -> [Gedung F]
Gedung H -> [Gedung E]
Gedung J -> [Gedung G]
Gedung A -> [Gedung B, Gedung C]
Gedung C -> [Gedung A, Gedung F]
Gedung B -> [Gedung A, Gedung D, Gedung E]
Gedung E -> [Gedung B, Gedung H]
Gedung D -> [Gedung B, Gedung G]
Gedung G -> [Gedung D, Gedung J]
Gedung F -> [Gedung C, Gedung I]

Hasil Pencarian

Jalur: [Gedung A, Gedung C]

Node Dikunjungi: [Gedung A, Gedung C]

Jumlah Node Dikunjungi: 2

Pencarian melalui DFS